Coinbase — anti-bot / device-protection assessment (2026-10-31)

Subject: Coinbase's web surfaces and what actually protects them
Method: public-site analysis + runtime observation. All 3rd-party telemetry observed or fulfilled locally; no account, no auth, nothing forwarded.
Bonus: analyzed `chris124567/cfsolve` (a Rust Cloudflare-solver) to ground the "what makes a good blocker" question in the checks it must satisfy.

> **TL;DR — Coinbase does NOT ship in-house client fingerprinting.** The exchange app has *no* device-intel bundle (its `coinbase_device_id` cookie is analytics bucketing, its `fingerprint` refs are Sentry). What actually protects them: **Cloudflare at the edge** (JS challenge + Turnstile on www, `cf_clearance`-gated) and **HMAC/TOTP API auth** on the exchange. The lesson: the hardest blockers are the ones you *buy* (Cloudflare's transport+JS-execution verification), not the ones you build in-house.

---

1. The two surfaces

Surface	URL	Protection
Marketing	`www.coinbase.com`	**Cloudflare JS challenge** → Turnstile widget → `cf_clearance`
Exchange app	`exchange.coinbase.com`	**No client fingerprinting** — HMAC/TOTP API auth + analytics cookies

2. Exchange app — what's actually in the client

- **No device-intel SDK** (no PerimeterX/HUMAN/Signifyd/Castle-style bundle).
- `coinbase_device_id` cookie = **experimentation bucketing** (their Tracker schema explicitly says so), not a device proof.
- All 54 `fingerprint` refs = **Sentry** error-monitoring internals.
- `hardwareConcurrency` / `deviceMemory` / `platform` = standard analytics fields.
- **Auth = Coinbase Exchange API:** `CB-ACCESS-KEY/SIGN/PASSPHRASE/TIMESTAMP` (HMAC-SHA256 of the request) + TOTP/2FA for high-value actions — server-verified, not client-computed device claims.
- Telemetry → `as.coinbase.com/metrics` (analytics service).

Verdict: the client is clean by design. There's nothing to read/tamper because the "protection" isn't a client token — it's API auth + edge.

3. The www Cloudflare layer (where the real defense is)

`www.coinbase.com` serves a **Cloudflare JS challenge** (not a Turnstile *widget* — an `orchestrate/chl_page` interactive challenge):

```
/cdn-cgi/challenge-platform/h/g/orchestrate/chl_page  → challenge page
challenges.cloudflare.com/turnstile/v0/g/<id>/api.js  → Turnstile loader
  sitekey: 0x4AAAAAAADnPIDROrmt1Wwj
  → interactive challenge (image + JS-proof) → cf_clearance cookie
```

The `/cdn-cgi/challenge-platform/h/g/*` endpoints seen:

Endpoint	Role
`orchestrate/chl_page`	challenge page orchestrator
`fo/<ray>/<token>`	fingerprint/orchestration POST (the JS-proof)
`i/<ray>/<token>`	interaction/behavior payload
`pat/<ts>/<hash>`	pass/attempt token
`ci/<ts>/<hash>`	challenge image

The challenge is **CSP-nonce-gated** (`script-src 'nonce-…' … challenges.cloudflare.com`), loads its real logic in **`blob:` workers**, and mints `cf_clearance` on success. This is the classic JS-challenge that cfsolve targets.

4. cfsolve — what "hard" actually means (the learning artifact)

`chris124567/cfsolve` is a **pure-Rust Cloudflare challenge/Turnstile solver** (no browser). To work, it must faithfully reproduce, in a custom BoringSSL HTTP/2 client + a custom JS VM:

1. **TLS ClientHello (JA3/JA4)** — exact cipher list, extensions, order of a real browser profile (`profiles/safari-26-ios.json`).
2. **HTTP/2 settings + header order** — including `sec-fetch-*` on each resource type (navigate/iframe/script) and exact `accept` / `accept-encoding`.
3. **JS execution fidelity** — it runs CF's own challenge script in a VM and reconstructs:
4. **`new Error()` call stacks** (`stage2/call_stack.rs`) — the challenge checks the *shape* of stack traces produced inside its own script.
5. **WASM benchmark timings** (`stage2/benchmark.rs`) — device CPU/GPU speeds measured via WebAssembly kernels.
6. **worker timing** (`stage2/worker_timing.rs`), **records/probes** (`stage2/records.rs`) — per-operation timing envelopes with exact field names.
7. **Proof + payload recovery** (`stage2/proof.rs`, `payload/recover.rs`) — the challenge response must round-trip the recorded probe values.

Why this is the "good" kind of blocker: it verifies the *entire transport + execution environment*, not a client token. A JS SDK (Stytch/Castle/etc.) ships its decision inputs to the client — readable/tamperable. Cloudflare's challenge verifies things the client can't fake without reimplementing a browser byte-for-byte: TLS fingerprint, HTTP/2 settings, header shapes, and the timing/stack behavior of a real JS engine. That's why even a dedicated solver like cfsolve needs a per-browser *profile* (recorded from a real browser + manual effort) — it's replaying a specific machine's verified fingerprint, and CF can rotate/check against drift.

5. The lessons (what makes a good blocker vs a bad one)

Bad (client-mintable)	Good (server-verified)
Ship decision inputs to the client (Stytch/Oscilar/Verisoul/Castle/SEON payloads)	Verify at the edge (Cloudflare TLS+HTTP2+JS execution)
Client mints the proof (all the SDKs we cracked)	Server issues the proof (cf_clearance after challenge)
Obfuscation = cost-raiser only (base91, XOR, VM — all RE-able)	Fidelity = real requirement (byte-exact transport + timing)
One bundle, deterministic	Per-browser profiles, drift-checked, rotating

Coinbase's take: they don't reinvent the hard part. Exchange = API auth (HMAC/TOTP, server-side). www = Cloudflare (the hard commodity). The "in-house" fingerprint you might expect doesn't exist because *the hard part isn't a client fingerprint — it's edge verification + server-side auth*.

6. Honest boundaries

- We did **not** solve the Cloudflare challenge or obtain a `cf_clearance` / logged-in session — we observed the flow and identified the Turnstile sitekey
- endpoint map (cfsolve is the tool that *would* solve it; running it on Coinbase is out of scope here).
- Exchange app = public bundle + unauthenticated page only; no account/auth touched. Everything observed or locally fulfilled.
- cfsolve was cloned/read for architectural analysis; not compiled or run.

7. Artifacts

File	What
`assets/exchange.index.js`	the 5.2 MB exchange bundle (from CDN)
`assets/turnstile.js`	Cloudflare Turnstile loader (86 KB)
`scripts/cb-probe.mjs`	exchange app probe (requests, globals, cookies)
`scripts/cb-www.mjs`	www Cloudflare challenge flow capture
`artifacts/probe1.json`	exchange request map + state
`artifacts/www-challenge.json`	the CF challenge endpoint flow
`cfsolve/` (cloned)	the Rust Cloudflare-solver (reference)

8. Summary line

> *"Coinbase doesn't do in-house device fingerprinting — the exchange app is a clean client with HMAC/TOTP API auth, and www sits behind Cloudflare's JS challenge (the hard kind: transport + JS-execution verification, which even a dedicated Rust solver like cfsolve has to profile per-browser). The best blockers verify the environment at the edge; the worst ship their decision inputs to the client."*